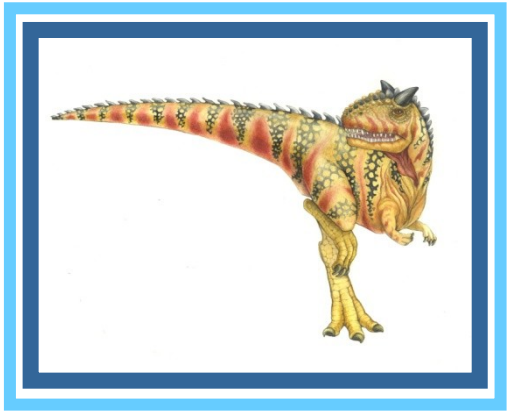
Chapter 11: File-System

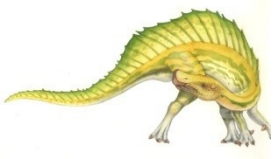


Objectives

- To explain the function of file systems
- To describe the interfaces to file systems
- To discuss file-system design

File System

- File system is the most visible aspect of an operating system. It provides the mechanism for on-line storage of and access to programs and data. It provides the persistent storage capability to a system.
- File systems consists of a collection of **files,** a **directory structure, access methods, secondary storage management and partitions** (which separate logical and physical collection of directories.)

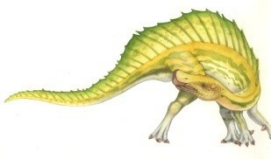


File Types – Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

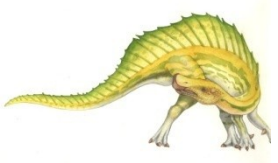
File Attributes

- **Name** – only information kept in human-readable form
- **Identifier** – unique tag (number) identifies file within file system
- **Type** – needed for systems that support different types
- **Location** – pointer to file location on device
- **Size** – current file size
- **Protection** – controls who can do reading, writing, executing
- **Time, date, and user identification** – data for protection, security, and usage monitoring
- Information about files are kept in the directory structure, which is maintained on the disk
- Many variations, including extended file attributes such as file checksum
- Information kept in the directory structure



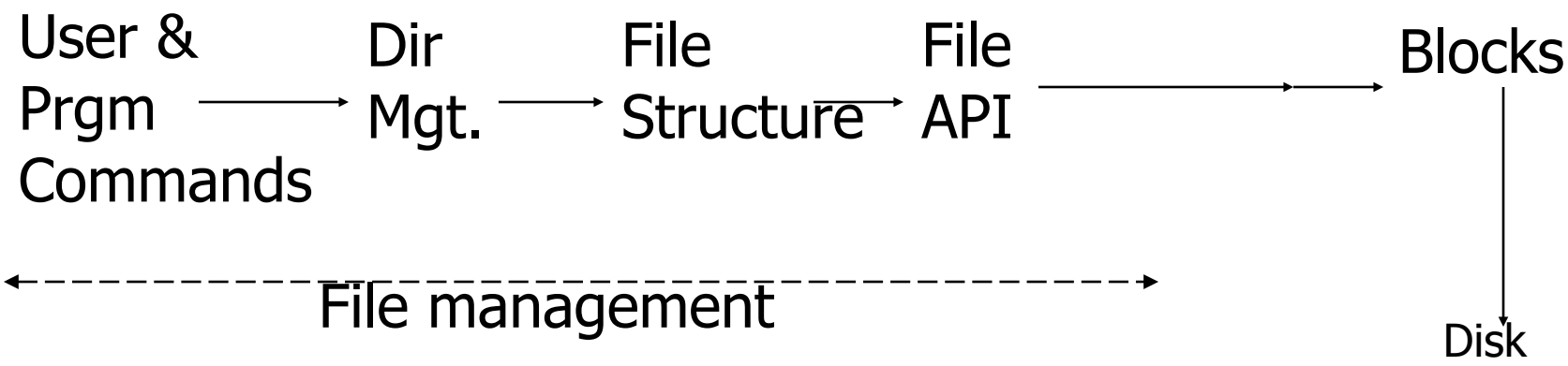
File Operations

- File is an **abstract data type**
- **Create**
- **Write –** at **write pointer** location
- **Read –** at **read pointer** location
- **Reposition within file - seek**
- **Delete**
- **Truncate**
- ***Open(F_i)*** – search the directory structure on disk for entry ***F_i***, and move the content of entry to memory
- ***Close (F_i)*** – move the content of entry ***F_i*** in memory to directory structure on disk

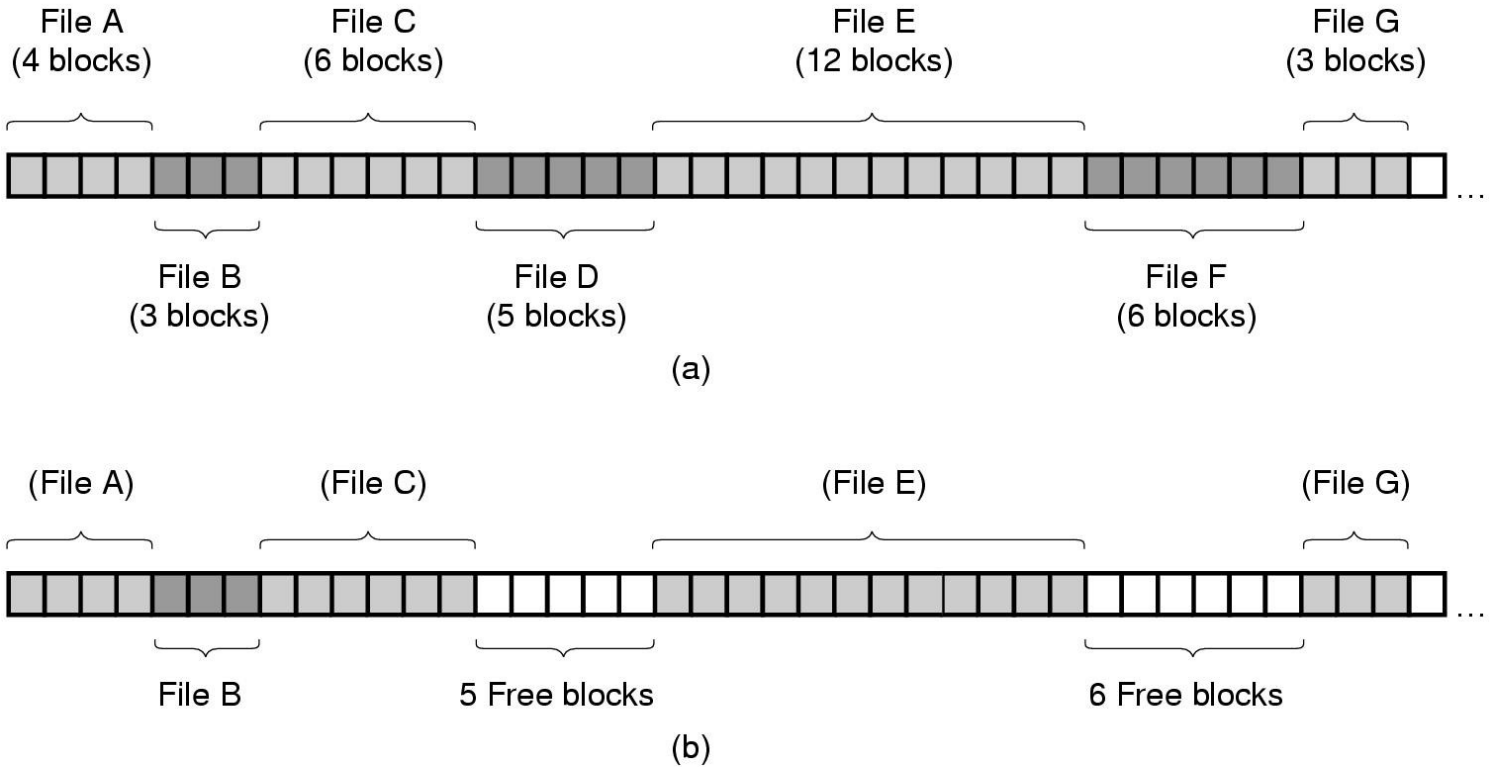


File Management

- Users and application programs interact with file system by means of commands for performing operations on files.
- These commands are translated into specific file manipulation commands, after ensuring that the kind of access requested is allowed.
- User view may be that of records or few bytes, but the actual IO is done in blocks. Data conversion to block "packing" is done. Optimized where applicable.
- Now IO subsystems takes over by translating the file sub commands into IO subsystem (disk IO) commands.



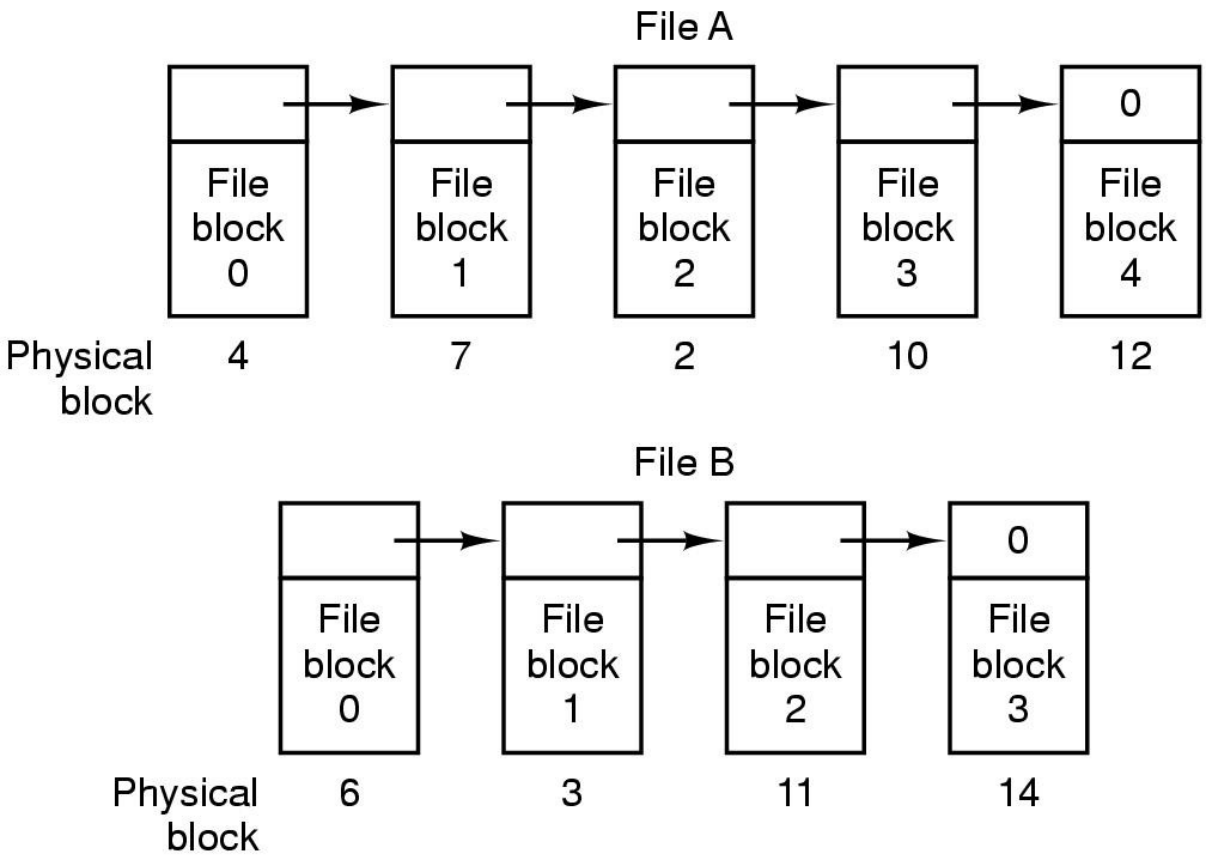
Implementing Files (1)



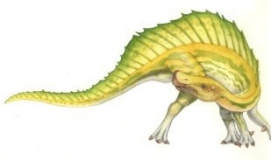
(a) Contiguous allocation of disk space for 7 files

(b) State of the disk after files *D* and *E* have been removed

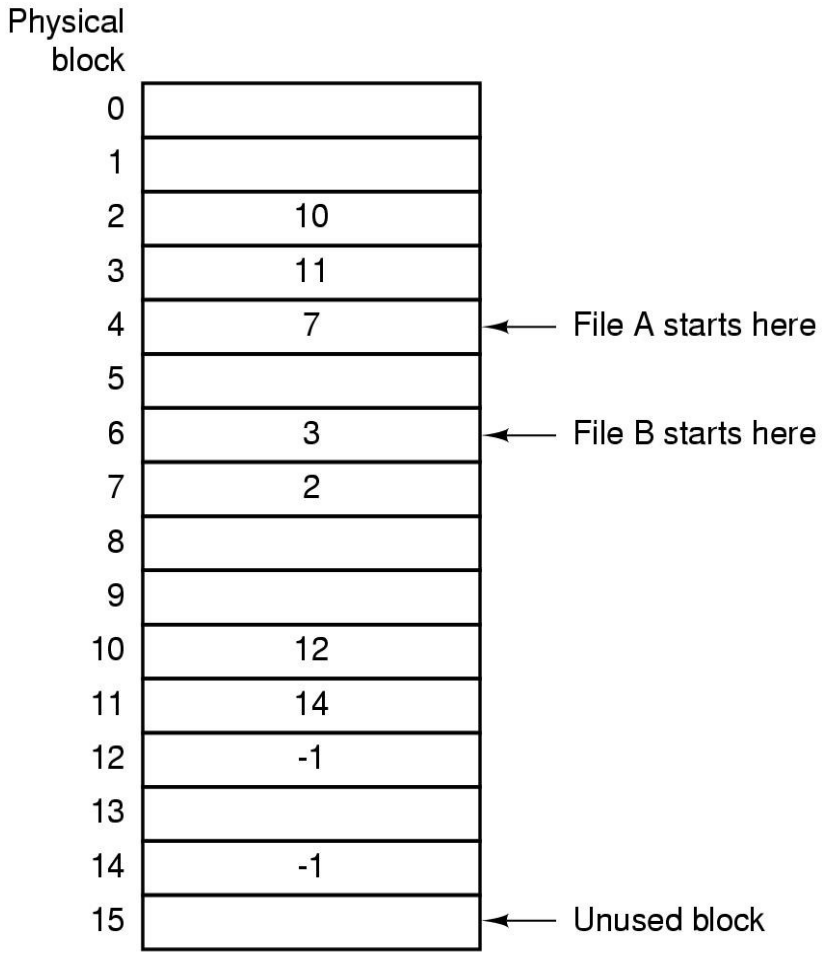
Implementing Files (2)



Storing a file as a linked list of disk blocks



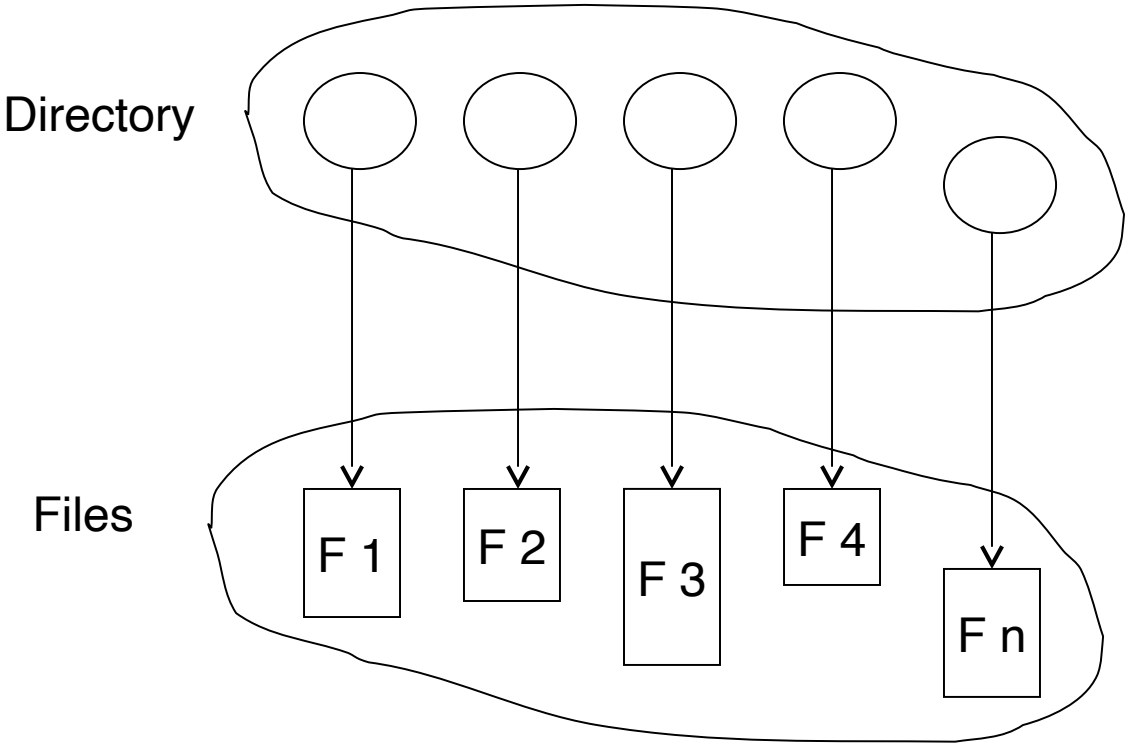
Implementing Files (3)



Linked list allocation using a file allocation table in RAM

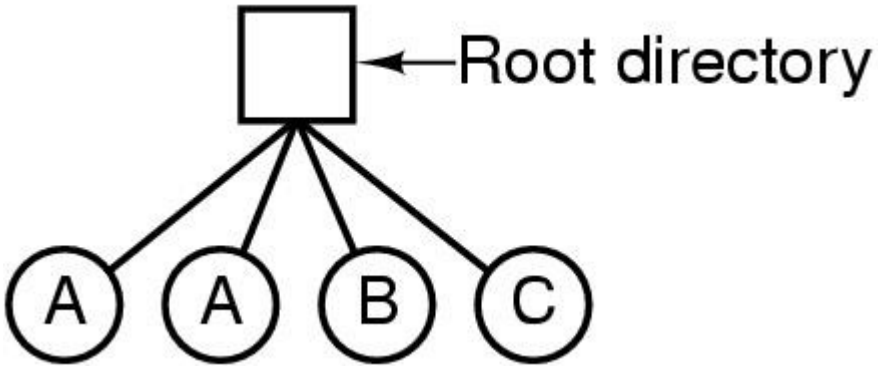
Directory Structure

- A collection of nodes containing information about files

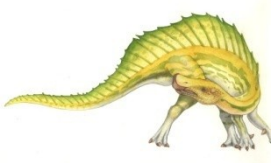


Both the directory structure and the files reside on disk

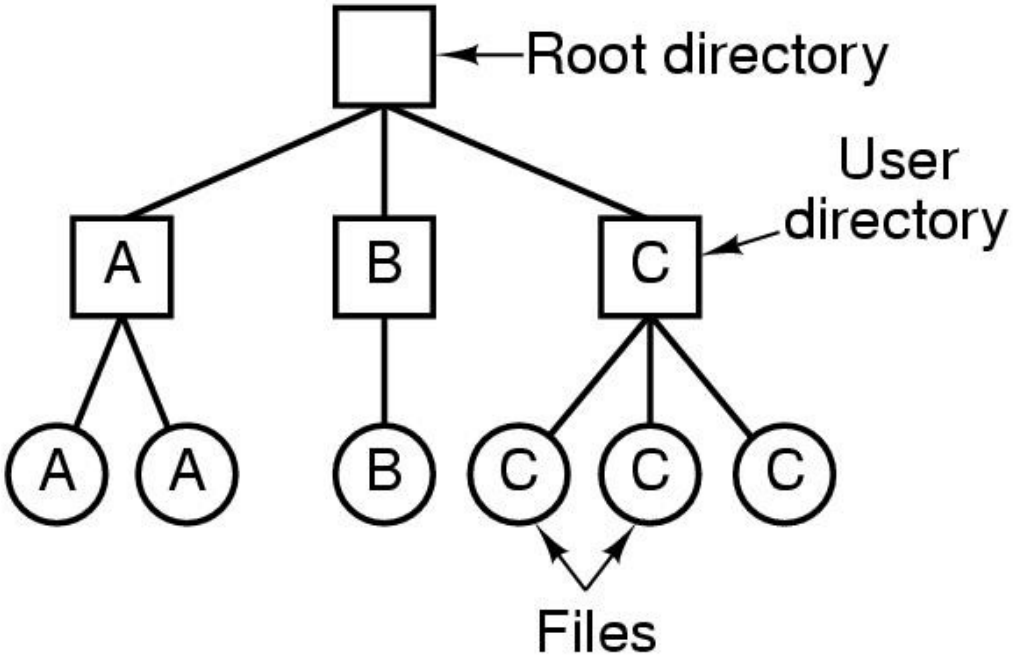
Single-Level Directory Systems



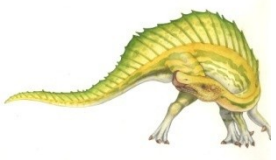
- A single level directory system
  - contains 4 files
  - owned by 3 different people, A, B, and C



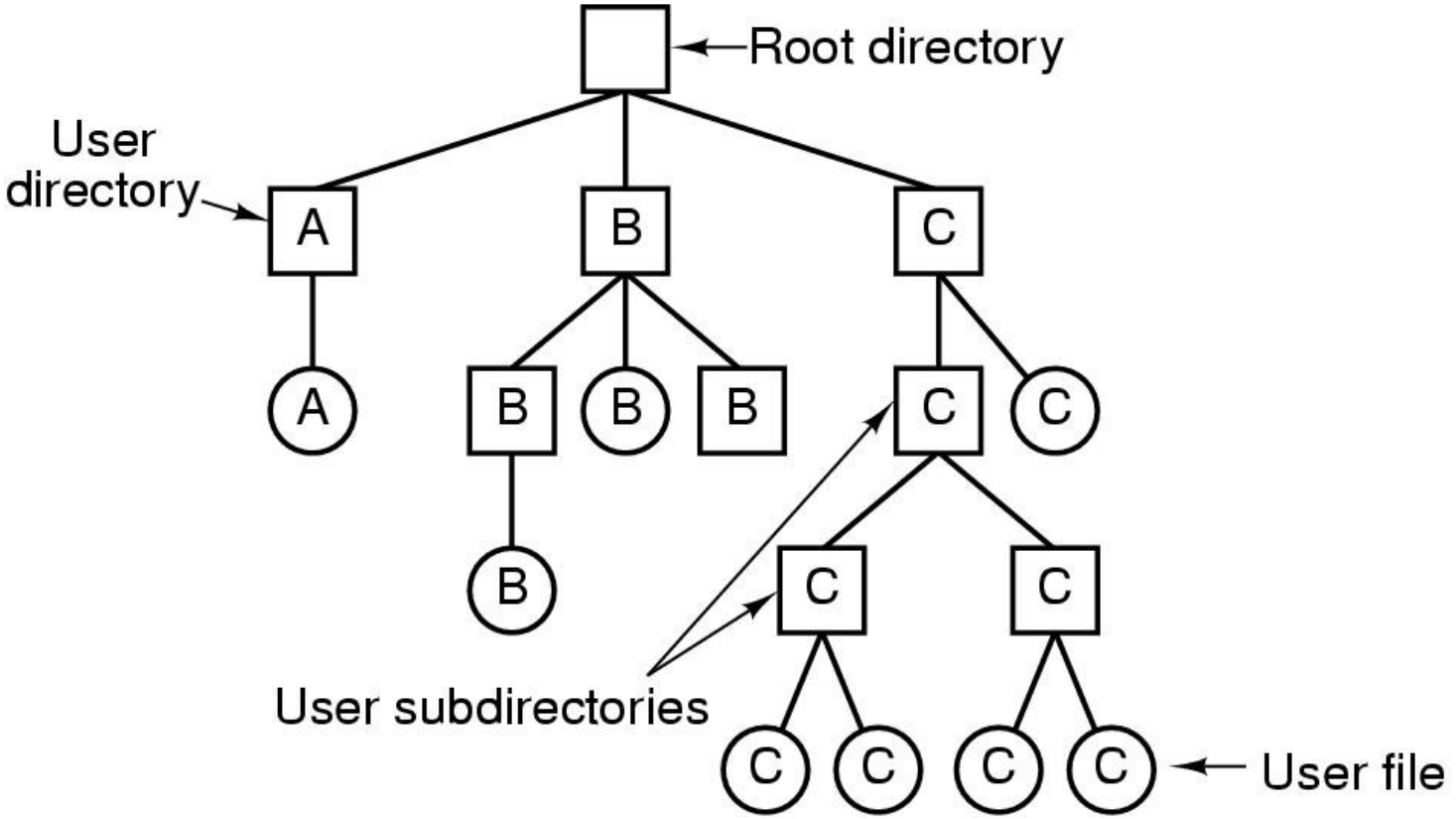
Two-level Directory Systems



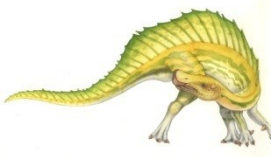
Letters indicate *owners* of the directories and files



Hierarchical Directory Systems



A hierarchical directory system



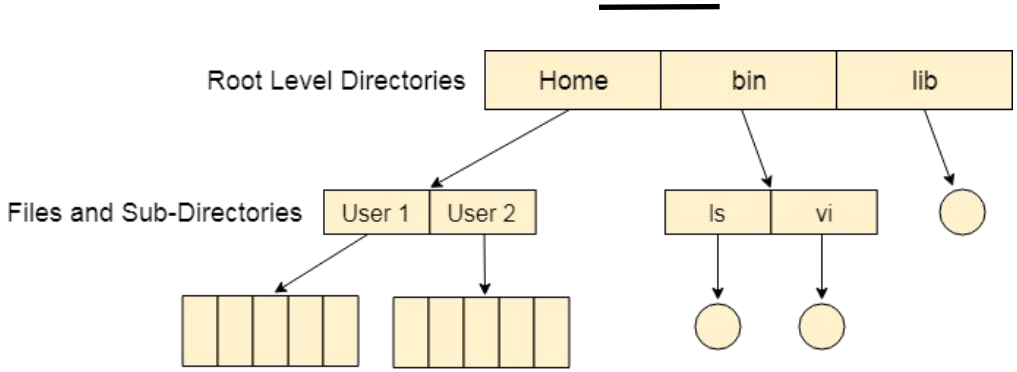
Directory Operations

1. Create
2. Delete
3. Opendir
4. Closedir
5. Readdir
6. Rename
7. Link
8. Unlink

Implementing Directories

games	attributes
mail	attributes
news	attributes
work	attributes

(a)



The Structured Directory System

(b)

(a) A simple directory

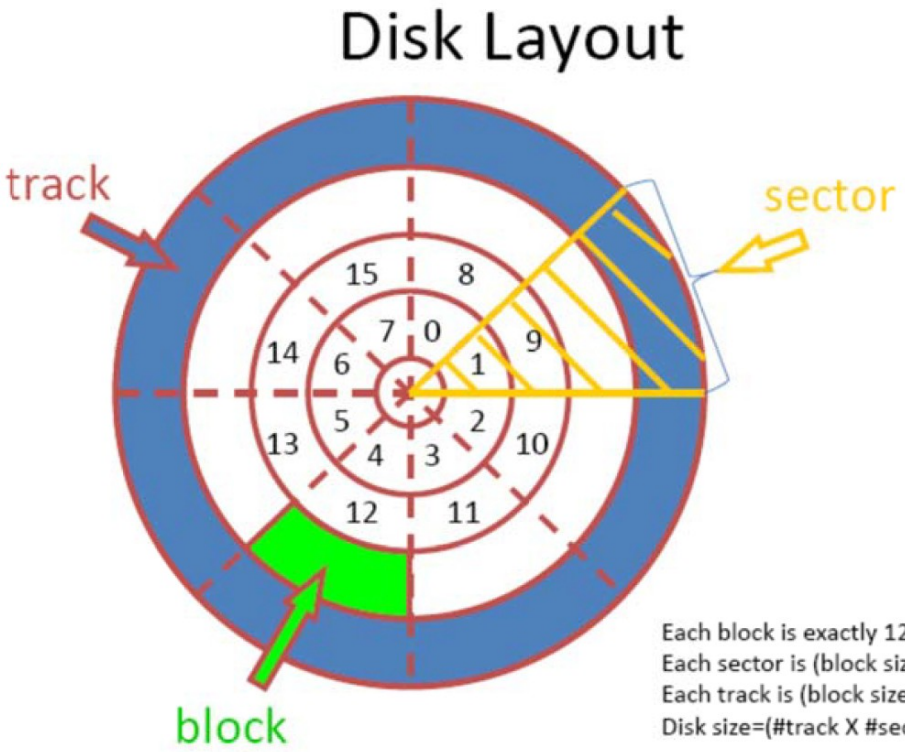
fixed size entries

disk addresses and attributes in directory entry

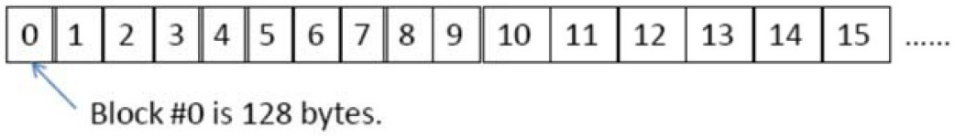
(b) Tree structure

Project 3 Disk Layout

- Chop the file into multiple blocks, and give each block an index (0, 1, …., n)
- Based on the cylinder ‘c’ and sector ‘s’, to calculate the block index is : block index = (c-1) * (# of sectors on the disk) + s
- For example, R 10 6. If you set the # of sectors on the disk to be 8 as shown in the figure on the left, then the block you are going to read is 10 * 8 + 6 = 86, which is #86 block



Each block is exactly 128 bytes.
Each sector is (block size X # of tracks) bytes.
Each track is (block size X # of sectors) bytes.
Disk size=(#track X #sect X block size) bytes.



Project 3 Disk Protocol

- The server must understand the following commands, and give the following responses:

- • I: information request. The disk returns two integers representing the disk geometry: the number of cylinders, and the number of sectors per cylinder.
- • R *c s*: read request for the contents of cylinder *c* sector *s*. The disk returns ’1’ followed by those 128 bytes of information, or ’0’ if no such block exists. (This will return whatever data happens to be on the disk in a given sector, even if nothing has ever been explicitly written there before.)
- • W *c s l* data: write request for cylinder *c* sector *s*. *l* is the number of bytes being provided, with a maximum of 128. The data is those *l* bytes of data. The disk returns ’1’ to the client if it is a valid write request (legal values of *c*, *s* and *l*), or returns a ’0’ otherwise. At what point in time this return code is sent, depends on the setting of the S synchronization mode. In cases where *l* < 128, the contents of those bytes of the sector between byte *l* and byte 128 is undefined—you may implement it any way that you want (for example: old data; zero-fill; etc.).

Project 3 Disk Protocol (cont)

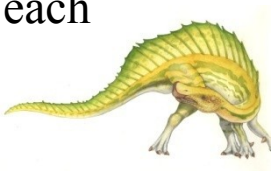
The data format that you *must* use for *c s* and *l* above is a regular ASCII string, followed by a white-space (space, tab or newline) character. So, for example, a read request for the contents of sector 17 of cylinder 130 would look like: R 130 17 . And then 129 bytes of data would be returned: the character 1, followed immediately by the 128 bytes read from that sector.

Project 3 Disk Client

The clients that you need to write here are mostly for testing purposes. The real use of this "disk" will be the filesystem implemented in the later parts of this project. So, for testing/demonstration purposes, you need to implement two clients:

(a)Command-line driven client: This client should work in a loop, having the user type commands in the format of the above protocol, send the commands to the disk server, and display the results to the user. (It would be difficult to debug your disk server without such a debugging tool anyway.)

(b)Random data client: This client should query the disk for its size (using the `I` command), and then generate a series of *N* random requests to the disk. Each request should randomly be `W` or `R`, and the cylinder number and sector number should be randomly chosen from the valid range for that disk. Write requests should all write 128 bytes of data. The value of *N*, and the seed for the random number generator, should be specified on the command line. This client must not print out the data that is read/written - simply printing a single character representing each request (to show progress) is sufficient.



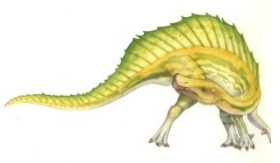
Project 3 File Operations

The server must understand the following commands, and give the following responses. (See the Appendix for some *suggested, but not required,* algorithms to implement some of these operations.)

- C *f*: create file.
- D *f*: delete file.
- L *b*: directory listing.
- R *f*: read file.
- W *f* *l* data: write file.

Project 3 Directory Operations

- mkdir *dirname* : create a directory of name *dirname*.
- cd *dirname* : change current working directory to *dirname*
- pwd : print the working directory name.
- rmdir *dirname*: remove the directory given. Throw an error if it is not present.



End of Chapter 11